Probabilistic Data Structures

a little error, a lot less memory

Watch the 4-min video

In one line: probabilistic structures trade a small, **bounded** error for fixed and tiny memory: a **Bloom filter** answers "definitely not" or "maybe" for set membership, a **cuckoo filter** does the same with deletes, **HyperLogLog** counts distinct items and a **Count-Min Sketch** estimates frequencies.

1 A billion keys, mostly missing

A database stores a billion keys on disk. Most lookups are for keys that don't exist (cache misses, existence checks, joins), and each one costs disk reads just to learn "not here". Keeping all keys in memory would take tens of gigabytes. A structure of ~1.2 GB that says "definitely not here" for 99% of absent keys removes almost all of those reads.

2 Bloom filters

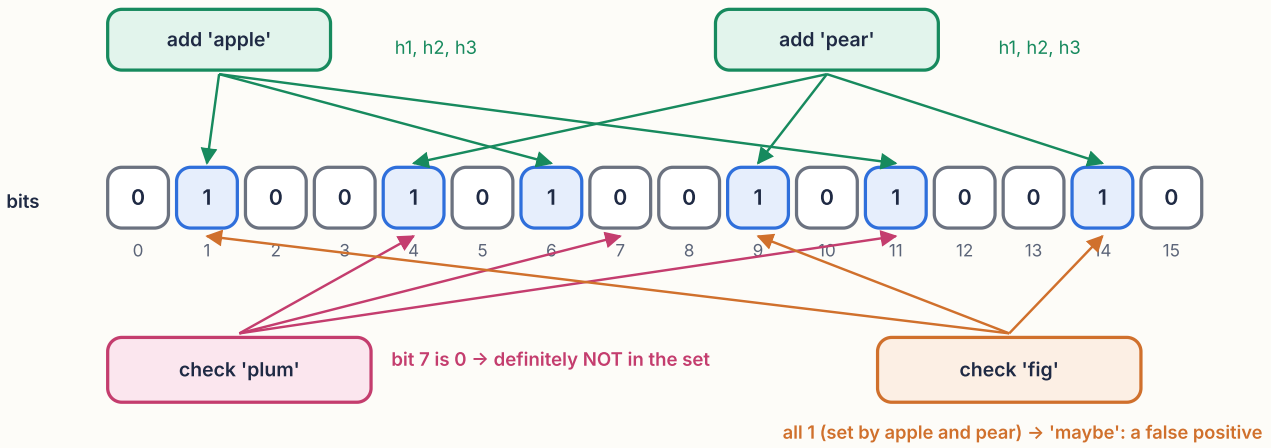


Figure 1. A 16-bit Bloom filter with k = 3 hash functions. A zero bit proves absence; all ones only suggests presence.

A Bloom filter (Burton Bloom, 1970) is an array of *m* bits and *k* hash functions. **Add:** set the *k* bits the item hashes to. **Check:** if any of those bits is 0, the item was never added; if all are 1, it *probably* was. So there are **no false negatives**, only false positives, and the rate is chosen by sizing:

$$\text{FPR} \approx (1 - e^{-kn/m})^k \quad \cdot \quad \text{best } k = (m/n)\cdot \ln 2 \quad \cdot \quad m/n \approx -1.44\cdot \log_2(\text{FPR})$$

Bits per item (m/n)	Best k	False-positive rate	1 billion items
4.8	3	≈ 10%	0.6 GB
9.6	7	≈ 1%	1.2 GB
14.4	10	≈ 0.1%	1.8 GB
19.2	13	≈ 0.01%	2.4 GB

Each extra ~4.8 bits per item cuts false positives 10×, and the item's own size doesn't matter: a 1 KB URL costs the same 10 bits as a user ID. In practice the k hashes come from two base hashes ($h_1 + i \cdot h_2$). Limits: you **can't delete** (clearing a bit might erase other items; counting Bloom filters use small counters instead of bits, at ~4× the memory), you **can't list** the items, and the false-positive rate rises if you add more items than planned, so size for growth or chain new filters (scalable Bloom filters).

3 Where Bloom filters earn their keep

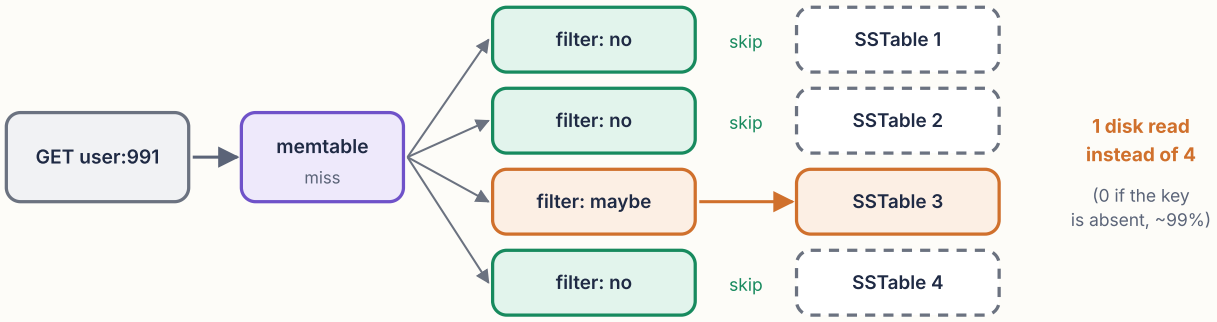


Figure 2. An LSM-tree read: each on-disk file has its own Bloom filter, kept in memory, so files that can't contain the key are never read.

LSM-tree databases (RocksDB, Cassandra, HBase, LevelDB) write data into many immutable files. A point lookup could have to check every file; a per-file Bloom filter turns most of those into memory checks. RocksDB users commonly configure about 10 bits per key (~1% FPR); Cassandra exposes the target directly as the table option `bloom_filter_fp_chance`. Other uses: CDNs and caches avoiding lookups for objects never seen, "is this username possibly taken?" before a database query, a feed's seen-items filter (lesson 29), and distributed joins that ship a filter instead of the key set.

4 Cuckoo filters: Bloom with deletes

A cuckoo filter (Fan et al., 2014) stores a short **fingerprint** (e.g. 8–16 bits) of each item in a table of buckets, typically 4 slots each. Every item has two candidate buckets: $i_1 = hash(x)$ and $i_2 = i_1$ *XOR* $hash(fingerprint)$, so either can be computed from the other plus the fingerprint. If both are full, an existing fingerprint is kicked to its alternate bucket, like a cuckoo chick. **Delete** removes a matching fingerprint. Lookups check just two buckets, and at low false-positive rates (below a few percent) it uses less space than a Bloom filter. Downsides: inserts can fail once the table is about 95% full, and deleting an item that was never added can remove another item's fingerprint.

5 The family

Structure	Question	Error	Memory	Deletes / merge
Bloom filter	is x in the set?	false positives only	~10 bits/item for 1%	no deletes; union = OR
Cuckoo filter	is x in the set?	false positives only	fingerprint + overhead per item	deletes yes
HyperLogLog	how many distinct?	±0.81% (12 KB, Redis)	fixed 12 KB	merge yes (lesson 38)
Count-Min Sketch	how often is x?	overestimates only, ≤ εN with prob. 1 − δ	w = ⌈e/ε⌉ × d = ⌈ln 1/δ⌉ counters	merge by adding grids
t-digest / KLL	what is the p99?	small rank error	a few KB	merge yes

For a Count-Min Sketch with ε = 0.1% and δ = 1%: width ⌈2.718/0.001⌉ = 2,719 and depth ⌈ln 100⌉ = 5, so 13,595 counters (~54 KB with 4-byte counters) estimate the frequency of any item to within 0.1% of the total count, with 99% probability, however many distinct items there are. It powers heavy-hitter detection in lesson 40. Redis Stack (RedisBloom) offers all of these: `BF.*`, `CF.*`, `CMS.*`, `TOPK.*`, plus built-in `PF*` for HyperLogLog.

WORKED EXAMPLE: SKIPPING DISK READS FOR MISSING KEYS

A Cassandra node holds 200 million rows spread over 40 SSTables. A `GET user:991` misses the memtable; the node checks the in-memory Bloom filter of each SSTable whose key range could contain it. 39 say "no" and are skipped without touching disk; 1 says "maybe" and is read. With a 1% false-positive rate, a lookup for an absent key reads a needless file only about once per 100 filters consulted, so 99 out of 100 useless reads disappear. The filters cost about 10 bits × 200M = 250 MB of RAM for the whole node.

TRADE-OFFS

- More bits per item: fewer false positives, more memory (≈4.8 bits per 10× improvement).
- Bloom: simplest and fastest, no deletes. Cuckoo: deletes and better space at low FPR, but inserts can fail when full.
- Sketches answer one question each; you can't recover the items or ask a different question later.
- Errors are bounded only if you size for the real item count; overfilled filters degrade quickly.

COMMON MISTAKES

- Using a "maybe" answer for permissions, billing or fraud blocks without an exact follow-up check.
- Trying to delete from a standard Bloom filter.
- Sizing for today's item count, then inserting ten times as many.
- Thinking a Bloom filter can give false negatives: a "no" is always correct.

6 Key terms

Bloom filter: bit array + k hashes; no false negatives

False positive: 'maybe present' for an item never added

Cuckoo filter: fingerprints in 2 candidate buckets; supports delete

Count-Min Sketch: d × w counters; frequency, overestimates only

Fingerprint: few-bit hash stored instead of the item

Sketch: small summary answering one question with bounded error

7 Interview questions

Q1. When would you use a Bloom filter? BASIC

For a fast, memory-cheap membership test where an occasional false positive is acceptable, such as skipping disk reads, cache checks or seen-item filters; a 'no' is always correct.

Q2. What's the general trade-off? BASIC

Bounded error in fixed, small memory: HyperLogLog for distinct counts, Count-Min Sketch for frequencies, cuckoo filters when you need deletes.

Q3. Size a Bloom filter for 100 million items at 1% false positives. SCENARIO

About 9.6 bits per item with 7 hashes: ~960 million bits ≈ 120 MB.

Q4. Users must be able to 'unsee' items in a seen filter. Which structure? SCENARIO

A cuckoo filter (or a counting Bloom filter), since standard Bloom filters can't delete.

Q5. Can a Bloom filter ever say 'not present' for an item that was added? TRICKY

No: adding sets all k bits and bits are never cleared, so a member always finds all ones. Only false positives are possible.

Q6. Why does a Count-Min Sketch only overestimate? TRICKY

Each counter for an item also absorbs collisions from other items, never subtracts; taking the minimum across rows picks the least polluted counter.

Next: SD-Q-015 · Top-K Heavy Hitters backendroadmap.com